1. **A symbol table stores information about identifiers (name, type, address) used during compilation. It supports insert, search, delete, and display operations.**

Source Code (s_table.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

typedef struct {
    char name[50];
    char type[20];
    int  address;
} Symbol;

Symbol table[MAX];
int count = 0;

void insert(char *name, char *type, int addr) {
    for (int i = 0; i < count; i++) {
        if (strcmp(table[i].name, name) == 0) {
            printf("Symbol '%s' already exists.\n", name);
            return;
        }
    }
    strcpy(table[count].name, name);
    strcpy(table[count].type, type);
    table[count].address = addr;
    count++;
    printf("Symbol '%s' inserted.\n", name);
}

int search(char *name) {
    for (int i = 0; i < count; i++)
        if (strcmp(table[i].name, name) == 0) return i;
    return -1;
}

void delete(char *name) {
    int pos = search(name);
    if (pos == -1) { printf("Symbol not found.\n"); return; }
    for (int i = pos; i < count - 1; i++) table[i] = table[i+1];
    count--;
    printf("Symbol '%s' deleted.\n", name);
}

void display() {
    printf("\n%-15s %-10s %s\n", "Name", "Type", "Address");
    printf("-----------------------------------\n");
    for (int i = 0; i < count; i++)
        printf("%-15s %-10s %d\n",
               table[i].name, table[i].type, table[i].address);
}

int main() {
    int choice, addr;
    char name[50], type[20];
```

```
    while (1) {
        printf("\n1.Insert 2.Search 3.Delete 4.Display 5.Exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Name Type Address: ");
                scanf("%s %s %d", name, type, &addr);
                insert(name, type, addr); break;
            case 2:
                printf("Name: "); scanf("%s", name);
                int r = search(name);
                if (r != -1) printf("Found at index %d\n", r);
                else printf("Not found.\n"); break;
            case 3:
                printf("Name: "); scanf("%s", name);
                delete(name); break;
            case 4: display(); break;
            case 5: exit(0);
        }
    }
}
```

Compilation Commands

```
gcc s_table.c
./a.out
```

Sample Input / Output

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 1
Name Type Address: x int 1000
Symbol 'x' inserted.

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 1
Name Type Address: y float 1002
Symbol 'y' inserted.

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 4
Name Type Address
---
x int 1000
y float 1002

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 2
Name: x
Found at index 0

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 2

Name: y
Found at index 1

1.Insert 2.Search 3.Delete 4.Display 5.Exit
Choice: 5

2. Develop a lexical analyzer to recognize a few patterns inc (ex. Identifiers, constants, comments, operators etc.)

Source Code (lexical.c)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char *word) {
    char *kw[] = {"int","float","char","if","else","while",
                  "for","return","void","do"};
    for (int i = 0; i < 10; i++)
        if (strcmp(kw[i], word) == 0) return 1;
    return 0;
}

void analyzeToken(char *token) {
    if (isKeyword(token))       printf("%-15s -> KEYWORD\n",
token);
    else if (isalpha(token[0])) printf("%-15s -> IDENTIFIER\n",
token);
    else if (isdigit(token[0])) printf("%-15s -> CONSTANT\n",
token);
    else                        printf("%-15s -> UNKNOWN\n",
token);
}

int main() {
    char line[200], token[50];
    int i = 0, j;
    printf("Enter C expression: ");
    fgets(line, 200, stdin);
    while (line[i] && line[i] != '\n') {
        while (line[i] == ' ') i++;
        if (line[i] == '/' && line[i+1] == '/')
            { printf("%-15s -> COMMENT\n", "//..."); break; }
        if (strchr("+-*/=<>!", line[i])) {
            char op[3] = {line[i], '\0', '\0'};
            if (line[i+1]=='='||
                (line[i]=='+'&&line[i+1]=='+')||
                (line[i]=='-'&&line[i+1]=='-'))
                { op[1]=line[++i]; }
            printf("%-15s -> OPERATOR\n", op);
            i++; continue;
```

```
        }
        if (strchr("();{}", line[i]))
            { printf("%-15c -> PUNCTUATION\n", line[i++]);
continue; }
        j = 0;
        while (isalnum(line[i]) || line[i]=='_')
token[j++]=line[i++];
        if (j) { token[j]='\0'; analyzeToken(token); }
        else if (line[i]) i++;
    }
    return 0;
}
```

Compilation Commands

```
gcc lexical.c
./a.out
```

Sample Input / Output

Enter C expression: **int x = 10 + y;**

int -> KEYWORD
x -> IDENTIFIER
= -> OPERATOR
10 -> CONSTANT
+ -> OPERATOR
y -> IDENTIFIER
; -> PUNCTUATION

3. Implementation of lexical analyzer using lex tool.

Lex Specification (lexer.l)

```
%{
#include <stdio.h>
%}

%%
"//"[^\n]*                  { printf("COMMENT: %s\n", yytext); }
"int"|"float"|"char"|"if"|"else"|"while"|"for"|"return"
                            { printf("KEYWORD: %s\n", yytext); }
[0-9]+\.[0-9]+              { printf("FLOAT_CONST: %s\n", yytext); }
[0-9]+                      { printf("INT_CONST: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*    { printf("IDENTIFIER: %s\n", yytext); }
"=="| "!="| "<="| ">="    { printf("RELOP: %s\n", yytext); }
[+\-*\/=<>!]               { printf("OPERATOR: %s\n", yytext); }
[();{},]                    { printf("PUNCTUATION: %s\n", yytext); }
[ \t\n]+                    { /* skip whitespace */ }
.                           { printf("UNKNOWN: %s\n", yytext); }
%%

int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

Compilation Commands

```
lex lexer.l
gcc lex.yy.c -o lexer
./lexer
```

Sample Input / Output

Input	int x = 10;

```
KEYWORD: int
IDENTIFIER: x
OPERATOR: =
INT_CONST: 10
PUNCTUATION: ;
```

4a) Recognize Valid Arithmetic Expression (+, -, *, /)

Lex File (arith.l)

```
%{
#include "y.tab.h"
%}
%%
[a-zA-Z]+   return VARIABLE;
[0-9]+      return NUMBER;
[\t]        ;
[\n]       return 0;
.          return yytext[0];
%%
int yywrap() { return 1; }
```

YACC File (arith.y)

```
%{
#include <stdio.h>
void yyerror(char *s);
%}
%name my_name_parser
%token NUMBER
%token
%left '+' '-'
%left '*' '/'
%left  '(' ')'

%%
expr : expr '+' expr
     | expr '-' expr
     | expr '*' expr
     | expr '/' expr
     | '(' expr ')'
     | NUM       { printf("\nEntered arithmetic expression is Valid\n\n");
                   return 0;
                 }

%%
int main() {
            printf("\nEnter Any Arithmetic Expression:\n");
               yyparse();
      }
void yyerror(char *s)
      { printf("INVALID EXPRESSION ERROR  ");
      }
```

Compilation Commands

```
yacc -d sample.y
lex sample.l
gcc lex.yy.c y.tab.c
./a.out
```

Sample input and output

Input	3+5*2

```
Valid arithmetic expression accepted.
```

4 b) Program to recognize a valid variable which starts with a letter followed by any number of letter or digits.

Lex File (variable.l)

```
%{
#include "y.tab.h"
%}
%%

[a-zA-Z]  { return LETTER; }
[0-9]     { return DIGIT; }
\n        { return 0; }
[ \t]     {}

%%
int yywrap() { return 1; }
```

YACC File (variable.y)

```
/* variable.y */
%{
#include <stdio.h>
void yyerror(char *s);
%}
%name my_name_parser
%token LETTER DIGIT
%%

variable : LETTER rest ;
rest     : rest LETTER | rest DIGIT | ;
%%

int main() {
      return yyparse(); }
void yyerror(char *s){
       printf("Invalid: %s\n",s); }
```

Compilation Commands

```
yacc -d variable.y
lex variable.l
gcc lex.yy.c y.tab.c
./a.out
```

Input	abc123 → Valid	123abc → ERROR

4c) Calculator Using Lex and YACC

Implementation of calculator using lex and yacc.

Lex File (calc.l)

```
%{
#include "y.tab.h"
extern int yylval;
%}

%%
[0-9]+            { yylval = atoi(yytext); return NUMBER; }       //
Recognize numbers
[  \t]+            ;     // Ignore whitespace
\n          { return 0; }     // End of input
.           { return yytext[0]; }   // Return single-character operators
%%
int yywrap()
{
return 1;
}
```

YACC File (calc.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(char *s);
%}
%name my_parser_name
%token NUMBER

// Operator precedence and associativity
%left  '+' '-'
%left  '*' '/'
%left  '(' ')'
%%
expression:
    E  {   printf("Result: %d\n", $1); }       // Print the final result
;
E   :      E '+' E   { $$ = $1 + $3; }
       |   E '-' E   { $$ = $1 - $3; }
       |   E '*' E   { $$ = $1 * $3; }
       |   E '/' E   {
                           if ($3 == 0) {
                                  yyerror("Division by zero");
                                  exit(1);
                                  }
                           $$ = $1 / $3;
                     }
       |   '(' E ')'        { $$ = $2; }
       |   NUMBER    { $$ = $1; }

%%

int main()
{
```

```
    printf("Enter an arithmetic expression: ");
    yyparse();
    return 0;
}

void yyerror( )
{
    printf( "Error: Invalid operations  \n");
}
```

Compilation Commands

```
yacc -d calc.y
lex calc.l
gcc lex.yy.c y.tab.c
./a.out
```

Sample Input / Output

```
Enter an arithmetic expression: 3-5*2
Result: -7
```

5.Converts BNF grammar rules into YACC form and generates an Abstract Syntax Tree (AST) for arithmetic expressions.

YACC File (ast.y)

```
%{
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    char op;  int val;  int isLeaf;
    struct Node *left, *right;
} Node;

Node* newLeaf(int v) {
    Node *n = malloc(sizeof(Node));
    n->isLeaf=1; n->val=v;
    n->left=n->right=NULL; return n;
}

Node* newNode(char op, Node *l, Node *r) {
    Node *n = malloc(sizeof(Node));
    n->isLeaf=0; n->op=op;
    n->left=l; n->right=r; return n;
}

void printAST(Node *n, int depth) {
    if (!n) return;
    for (int i=0; i<depth; i++) printf("  ");
    if (n->isLeaf) printf("%d\n", n->val);
    else           printf("%c\n", n->op);
    printAST(n->left,  depth+1);
```

```
    printAST(n->right, depth+1);
}

void yyerror(char *s){ printf("Error: %s\n",s); }
int yylex();
%}

%union { int ival; struct Node *node; }
%token <ival> NUM
%type  <node> expr term factor
%left '+' '-'
%left '*' '/'
%%
start  : expr { printf("AST:\n"); printAST($1, 0); }
       ;
expr   : expr '+' term { $$ = newNode('+',$1,$3); }
       | expr '-' term { $$ = newNode('-',$1,$3); }
       | term          { $$ = $1; }
       ;
term   : term '*' factor { $$ = newNode('*',$1,$3); }
       | term '/' factor { $$ = newNode('/',$1,$3); }
       | factor          { $$ = $1; }
       ;
factor : '(' expr ')'  { $$ = $2; }
       | NUM           { $$ = newLeaf($1); }
       ;
%%
int main() { return yyparse(); }
```

Compilation Commands

```
yacc -d ast.y
gcc  y.tab.c
./a.out
```

Sample Input / Output

Input	3+5*2

```
AST:
+
  3
  *
    5
    2
```

6. Checks type compatibility for arithmetic/logical operations on int, float, and char types.

Source Code (type_chk.c)

```
#include <stdio.h>
#include <string.h>

typedef enum { INT, FLOAT, CHAR, UNKNOWN } Type;

Type getType(char *t) {
    if (strcmp(t,"int")==0)   return INT;
    if (strcmp(t,"float")==0) return FLOAT;
    if (strcmp(t,"char")==0)  return CHAR;
    return UNKNOWN;
}

Type checkArith(Type t1, Type t2) {
    if (t1==UNKNOWN || t2==UNKNOWN) return UNKNOWN;
    if (t1==FLOAT   || t2==FLOAT)   return FLOAT;
    if (t1==INT     && t2==INT)      return INT;
    return UNKNOWN;
}

int main() {
    char type1[10], type2[10], op[3];
    printf("Enter: type1 op type2 (e.g. int + float): ");
    scanf("%s %s %s", type1, op, type2);

    Type t1 = getType(type1), t2 = getType(type2);
    if (t1 == UNKNOWN) { printf("Unknown type: %s\n", type1); return 1; }
    if (t2 == UNKNOWN) { printf("Unknown type: %s\n", type2); return 1; }

    if ((t1==CHAR||t2==CHAR) && (op[0]=='*'||op[0]=='/')) {
        printf("TYPE ERROR: Cannot apply '%s' on char\n", op);
        return 1;
    }

    Type result = checkArith(t1, t2);
    char *rname = (result==INT)?"int":(result==FLOAT)?"float":"unknown";
    printf("Type check OK. Result type: %s\n", rname);
    return 0;
}
```

Compilation Commands

```
gcc type_chk.c
./a.out
```

Sample Input / Output

Input 1	int + float

```
Type check OK. Result type: float
```

Input 2	char * int

```
TYPE ERROR: Cannot apply '*' on char
```

7. Simulates stack-based storage allocation for variables in an activation record, tracking offsets and sizes.

Source Code (stack.c)

```
#include <stdio.h>
#include <string.h>

#define STACK_SIZE 1024

char stack[STACK_SIZE];
int  top = 0;

typedef struct {
    char name[20];
    int  offset;
    int  size;
} Variable;

Variable activationRecord[20];
int varCount = 0;

void enter_scope() {
    printf("Entering scope. Stack top: %d\n", top);
}

void allocate(char *name, int size) {
    if (top + size > STACK_SIZE) { printf("Stack overflow!\n"); return; }
    strcpy(activationRecord[varCount].name,   name);
    activationRecord[varCount].offset = top;
    activationRecord[varCount].size   = size;
    varCount++;
    top += size;
    printf("Allocated '%s': offset=%d, size=%d bytes\n",
            name, top-size, size);
}

void exit_scope(int saved_top) {
    top = saved_top;
    printf("Exiting scope. Stack top reset to: %d\n", top);
}

void display_stack() {
    printf("\n%-10s %-8s %s\n","Variable","Offset","Size");
    printf("----------------------------\n");
    for (int i=0; i<varCount; i++)
        printf("%-10s %-8d %d\n",
               activationRecord[i].name,
               activationRecord[i].offset,
               activationRecord[i].size);
    printf("Total stack used: %d bytes\n\n", top);
}

int main() {
    int saved;
    enter_scope();
    saved = top;
    allocate("x",   4);   /* int   */
```

```
    allocate("y",   4);   /* int   */
    allocate("arr", 20);  /* int[5]*/
    display_stack();
    exit_scope(saved);
    return 0;
}
```

Compilation Commands

```
gcc stack.c
./a.out
```

Sample Output

```
Entering scope. Stack top: 0
Allocated 'x':   offset=0,  size=4 bytes
Allocated 'y':   offset=4,  size=4 bytes
Allocated 'arr': offset=8,  size=20 bytes

Variable   Offset   Size
----------------------------
x          0        4
y          4        4
arr        8        20
Total stack used: 28 bytes

Exiting scope. Stack top reset to: 0
```

Program 8: Count Words and Lines Using Lex Tool

Lex Specification (wc.l)

```
%{
int words = 0, lines = 0;
%}
%%
\n            { lines++; }
[^ \t\n]+     { words++; }
[ \t]+        { }
%%
int main(int argc, char *argv[]) {
    if (argc > 1) yyin = fopen(argv[1], "r");
    yylex();
    printf("Lines: %d\nWords: %d\n", lines, words);
    return 0;
}
int yywrap() { return 1; }
```

Compilation Commands

```
lex wc.l
gcc lex.yy.c -o wc
./wc test.txt
```

Sample Input File (test.txt)

```
hello world
this is compiler design lab
```

Sample Output

```
Lines: 2
Words: 7
```

9. Write a ‘C’ program to implement lexical analyzer using c program.

A complete, production-quality lexical analyzer implemented entirely in C without any external tools. Handles all major token classes including multi-character operators, float constants, string literals, and single-line/block comments.

Source Code (lexical_analyzer.c)

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdlib.h>

#define MAX_TOKEN 100

typedef enum {
    TOK_KEYWORD, TOK_IDENTIFIER, TOK_INT_CONST, TOK_FLOAT_CONST,
    TOK_STRING, TOK_OPERATOR, TOK_RELOP, TOK_PUNCT,
    TOK_COMMENT, TOK_EOF, TOK_UNKNOWN
} TokenType;

typedef struct { TokenType type; char value[MAX_TOKEN]; } Token;

char *keywords[] = {"int","float","char","void","if","else","while",
                    "for","do","return","break","continue",NULL};

int isKeyword(char *s) {
    for (int i=0; keywords[i]; i++)
        if (strcmp(keywords[i],s)==0) return 1;
    return 0;
}

const char *typeName(TokenType t) {
    switch(t) {
        case TOK_KEYWORD:     return "KEYWORD";
        case TOK_IDENTIFIER:  return "IDENTIFIER";
        case TOK_INT_CONST:   return "INT_CONSTANT";
        case TOK_FLOAT_CONST: return "FLOAT_CONSTANT";
        case TOK_STRING:      return "STRING_LITERAL";
        case TOK_OPERATOR:    return "OPERATOR";
        case TOK_RELOP:       return "RELOP";
        case TOK_PUNCT:       return "PUNCTUATION";
        case TOK_COMMENT:     return "COMMENT";
        default:              return "UNKNOWN";
    }
}

Token getNextToken(const char *src, int *pos) {
    Token tok; tok.value[0]='\0';
    while (src[*pos]==' '||src[*pos]=='\t'||src[*pos]=='\n') (*pos)++;
    if (!src[*pos]) { tok.type=TOK_EOF; strcpy(tok.value,"EOF"); return
tok; }

    /* single-line comment */
    if (src[*pos]=='/'&&src[*pos+1]=='/') {
        tok.type=TOK COMMENT; int j=0;
        while (src[*pos]&&src[*pos]!='\n') tok.value[j++]=src[(*pos)++];
        tok.value[j]='\0'; return tok;
    }
```

```
    /* string literal */
    if (src[*pos]=='"'){
        tok.type=TOK_STRING; int j=0;
        tok.value[j++]=src[(*pos)++];
        while (src[*pos]&&src[*pos]!='"')
        { if(src[*pos]=='\\') tok.value[j++]=src[(*pos)++];
          tok.value[j++]=src[(*pos)++]; }
        if(src[*pos]) tok.value[j++]=src[(*pos)++];
        tok.value[j]='\0'; return tok;
    }

    /* number */
    if (isdigit(src[*pos])) {
        int j=0,dot=0;
        while (isdigit(src[*pos])||(src[*pos]=='.'&&!dot))
        { if(src[*pos]=='.') dot=1; tok.value[j++]=src[(*pos)++]; }
        tok.value[j]='\0';
        tok.type = dot ? TOK_FLOAT_CONST : TOK_INT_CONST; return tok;
    }

    /* identifier / keyword */
    if (isalpha(src[*pos])||src[*pos]=='_') {
        int j=0;
        while (isalnum(src[*pos])||src[*pos]=='_')
            tok.value[j++]=src[(*pos)++];
        tok.value[j]='\0';
        tok.type = isKeyword(tok.value)?TOK_KEYWORD:TOK_IDENTIFIER;
        return tok;
    }

    /* two-char operators */
    char c=src[*pos], nc=src[*pos+1];
    if((c=='='&&nc=='=')||(c=='!'&&nc=='=')||
       (c=='<'&&nc=='=')||(c=='>'&&nc=='=')||
       (c=='+'&&nc=='+')||(c=='-'&&nc=='-'))
    {
        tok.type=(c=='<'||c=='>'||c=='='||c=='!')?TOK_RELOP:TOK_OPERATOR;
        tok.value[0]=c; tok.value[1]=nc; tok.value[2]='\0';
        (*pos)+=2; return tok;
    }

    /* single-char operator */
    if (strchr("+-*/%=<>!&|^", c)) {
        tok.type=(c=='<'||c=='>'||c=='=')?TOK_RELOP:TOK_OPERATOR;
        tok.value[0]=c; tok.value[1]='\0'; (*pos)++; return tok;
    }

    /* punctuation */
    if (strchr("()[]{};,.", c)) {
        tok.type=TOK_PUNCT;
        tok.value[0]=c; tok.value[1]='\0'; (*pos)++; return tok;
    }

    tok.type=TOK_UNKNOWN;
    tok.value[0]=c; tok.value[1]='\0'; (*pos)++; return tok;
}
```

```
int main() {
    const char *src =
        "int main() {\n"
        "    int x = 10; // declare x\n"
        "    float y = 3.14;\n"
        "    if (x > 5) { x++; }\n"
        "    return 0;\n"
        "}";
    int pos = 0; Token tok;
    printf("%-20s %s\n","Token","Type");
    printf("----------------------------------------\n");
    do {
        tok = getNextToken(src, &pos);
        printf("%-20s %s\n", tok.value, typeName(tok.type));
    } while (tok.type != TOK_EOF);
    return 0;
}
```

Compilation Commands

```
gcc lexical_analyzer.c
./a.out
```

Sample Output

```
Token                Type
----------------------------------------
int                  KEYWORD
main                 IDENTIFIER
(                    PUNCTUATION
)                    PUNCTUATION
{                    PUNCTUATION
int                  KEYWORD
x                    IDENTIFIER
=                    RELOP
10                   INT_CONSTANT
;                    PUNCTUATION
// declare x         COMMENT
float                KEYWORD
y                    IDENTIFIER
=                    RELOP
3.14                 FLOAT_CONSTANT
;                    PUNCTUATION
if                   KEYWORD
x                    IDENTIFIER
>                    RELOP
5                    INT_CONSTANT
x                    IDENTIFIER
++                   OPERATOR
;                    PUNCTUATION
return               KEYWORD
0                    INT_CONSTANT
;                    PUNCTUATION
EOF                  EOF
```

10. write recursive descent parser for the grammar E->E+T E->T T->T*F T->F F->(E)/id.

Grammar: E→E+T | E-T | T, T→T*F | T/F | F, F→(E) | id

We must first eliminate **left recursion**, since recursive descent parsers **cannot handle left-recursive grammars**.

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

After removing left recursion

$E \rightarrow T\ E'$

$E' \rightarrow + T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow * F\ T' \mid \varepsilon$

$F \rightarrow (\ E\ ) \mid id$

Source Code (recursive_1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
char lookahead;     // Global variable to store the current
lookahead character
void match(char c);
void E();
void E_dash();
void T_dash();
void T();
void F();
// Function to consume the current lookahead character and get
the next one
void match(char c)
{
    if (lookahead == c)
   {
        lookahead = getchar();
    }
 else
 {
  printf("Error: Expected '%c', but found '%c'\n", c,
lookahead);
        exit(1);
  }
}
```

```
// Function for non-terminal F (Factor)       F → ( E ) | id

void F()
 {
    if (lookahead == '(')
 {
        match('(');
        E(); // Call E function for sub-expression
        match(')');
     }
else if (lookahead == 'i')
 {                   // 'i' represents an identifier/operand
                match('i');
     }
else
 {
        printf("Error: Invalid factor\n");
        exit(1);
     }
}

// Function for non-terminal T' (T-dash for handling
multiplication)
void T_dash()
{
    if (lookahead == '*')
 {
        match('*');
        F();
        T_dash(); // Recursively call T_dash for more
multiplications
          }
}

// Function for non-terminal T (Term)
void T()
 {
    F();
    T_dash();
}

// Function for non-terminal E' (E-dash for handling addition)
void E_dash()
{
    if (lookahead == '+')
  {
        match('+');
        T();
        E_dash(); // Recursively call E_dash for more
```

```
additions
    }
    // If no '+', it's an epsilon production (return)
}

// Function for non-terminal E (Expression)
void E()
{
    T();
    E_dash();
}

int main()
{
    printf("Enter an arithmetic expression (e.g., i+i*i$):
\n ");
    lookahead = getchar();    // Get the first character
    E();            // Start parsing from the start symbol E

    if (lookahead == '$')          // '$' typically marks the
end of input
    {
        printf("Parsing Successful!\n");
    }
  else
  {
        printf("Error: Extra characters after parsing\n");
    }

    return 0;
}
```

Compilation Commands

```
gcc recursive_1.c
./a.out
```

Sample Output

```
Enter an arithmetic expression (e.g., i+i*i$):
i+(i*i)$
Parsing Successful!

~$ ./a.out
Enter an arithmetic expression (e.g., i+i*i$):
 ++a
Error: Invalid factor
```

11. write recursive descent parser for the grammar S->(L) S->a L->L,S L->S

Grammar: S→(L) | a, L→L,S | S

```
Step 1:     Eliminate Left Recursion
            The non-terminal L has left recursion:
L → L , S | S
We'll rewrite it:
L → S L'
L' → , S L' | ε

Step 2: Final Grammar (No Left Recursion)
S  → ( L ) | a
L  → S L'
L' → , S L' | ε
```

Source Code (recursive_2.c)

```
#include <stdio.h>
#include <stdlib.h>
char lookahead;
void L();
void L_dash();
void S();
void match(char);

void match(char c)
{
    if (lookahead == c)
   {
        lookahead = getchar();
    }
 else
 {
        printf("Error: Expected '%c', but found '%c'\n", c, lookahead);
        exit(1);
  }
}
int main()
{
    printf("Enter a string  ends with $ ");
    lookahead = getchar();          // Get the first character
    S();          // Start parsing from the start symbol E
    if (lookahead == '$')
    {                   // '$' typically marks the end of input
        printf("Parsing Successful!\n");
    }
  else
  {
        printf("Error: Extra characters after parsing\n");
    }
    return 0;
}
// S → (L) | a
void S()
{
```

```
    if (lookahead == '(')
  {
        match('(');
        L();
        match(')');
    }
else if (lookahead == 'a')
{
        match('a');
    }
 else
{
        printf("Syntax Error in S: found '%c'\n", lookahead);
        exit(1);
    }
}
// L → S L'
void L()
{
    S();
    L_dash();
}

// L' → , S L' | ε
void L_dash()
{
    if (lookahead == ',')
   {
        Match(',');
        S();
       L_dash();
    }
 else
      return;
}
```

Compilation Commands

```
gcc recursive_2.c
./a.out
```

Sample Output

```
Enter a string  ends with $ (a)$
Parsing Successful!


~$ ./a.out
Enter a string  ends with $ (a())$
Error: Expected ')', but found '('
~$ ./a.out
Enter a string  ends with $ (a(a)$
Error: Expected ')', but found '(
```

12. Write a C program to calculate first function for the grammar E->E+T E->T T->T*F T->F F->(E)/id

Computes the FIRST set for each non-terminal symbol. After eliminating left recursion: E→TE', E'→+TE'|ε, T→FT', T'→*FT'|ε, F→(E)|id

Source Code (First.C)

```
#include <stdio.h>
#include <string.h>

#define MAXNT 10
#define MAXFIRST 10

typedef struct {
    char nt;
    char first[MAXFIRST][10];
    int  count;
} FirstSet;

void addFirst(FirstSet *f, const char *sym) {
    for (int i=0; i<f->count; i++)
        if (strcmp(f->first[i], sym)==0) return; /* no dup */
    strcpy(f->first[f->count++], sym);
}

void computeFirst(FirstSet sets[], int n) {
    /* F -> (E) | id  => FIRST(F) = { (, id } */
    /* T -> FT'       => FIRST(T) = FIRST(F)   */
    /* E -> TE'       => FIRST(E) = FIRST(T)   */
    addFirst(&sets[2], "(");   /* F */
    addFirst(&sets[2], "id");

    addFirst(&sets[1], "(");   /* T */
    addFirst(&sets[1], "id");

    addFirst(&sets[0], "(");   /* E */
    addFirst(&sets[0], "id");
}

int main() {
    FirstSet sets[3];
    char *names[] = {"E","T","F"};
    for (int i=0; i<3; i++) { sets[i].count=0; }

    computeFirst(sets, 3);

    printf("%-10s %s\n","Non-Term","FIRST Set");
    printf("-----------------------------------\n");
    for (int i=0; i<3; i++) {
        printf("%-10s { ", names[i]);
        for (int j=0; j<sets[i].count; j++) {
            printf("%s", sets[i].first[j]);
            if (j < sets[i].count-1) printf(", ");
        }
        printf(" }\n");
    }

    printf("\nFIRST(E+T) = FIRST(E) = { (, id }\n");
    printf("FIRST(+T)  = { + }\n");
    printf("FIRST((E)) = { ( }\n");
    printf("FIRST(id)  = { id }\n");
    return 0;
}
```

Compilation Commands

```
gcc First.c
./a.out
```

Sample Output

```
Non-Term   FIRST Set
-----------------------------------
E          { (, id }
T          { (, id }
F          { (, id }

FIRST(E+T) = FIRST(E) = { (, id }
FIRST(+T)  = { + }
FIRST((E)) = { ( }
FIRST(id)  = { id }
```